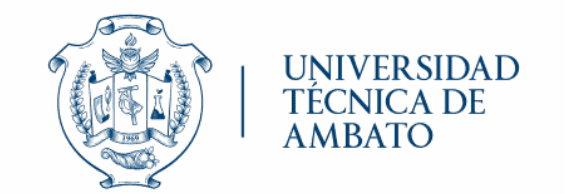
UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL
CARRERA DE INGENIERÍA DE SOFTWARE



CATÁLOGO DE LA BIBLIOTECA FISEI

Árbol general y árbol binario de búsqueda en Java

Asignatura	Estructura de Datos
Nivel	Tercero – Ingeniería de Software
Modalidad	Individual
Estudiante	Fernando Villacres
Docente	José Caiza
Fecha	22 de octubre de 2026

Informe de práctica

1. Presentación

En esta práctica se construyó una aplicación de consola para representar la organización de la Biblioteca FISEI y consultar sus libros. La biblioteca se organiza en áreas y estantes; dentro de cada estante, los libros se ordenan por su código. Para representar estas dos necesidades se implementaron un árbol general y un árbol binario de búsqueda, respectivamente.

Además de las operaciones básicas, el trabajo permite ver paso a paso los recorridos recursivos. Esto ayuda a relacionar el orden en que se ejecutan las llamadas con el resultado que aparece en pantalla.

2. Cómo se organizó el programa

El código se separó en paquetes para distinguir los tipos de unidad, las clases del modelo, la lógica de los árboles y el programa de consola. Cada estante crea su propio catálogo; las áreas y la biblioteca, en cambio, solo mantienen a sus hijos.

Paquete o clase	Función dentro del programa
enums	Define los tipos de unidad y de recorrido.
modelo.Libro	Guarda el código, título, autor y año, aplicando las validaciones.
modelo.UnidadBiblioteca	Representa una biblioteca, área o estante.
negocio.ArbolBiblioteca	Administra la jerarquía y sus consultas.
negocio.CatalogoABB	Administra los libros de un estante mediante un ABB.
app.Main y DatosPrueba	Presentan el menú y cargan los datos de ejemplo.
app.PruebasBiblioteca	Comprueba los casos de la consigna.

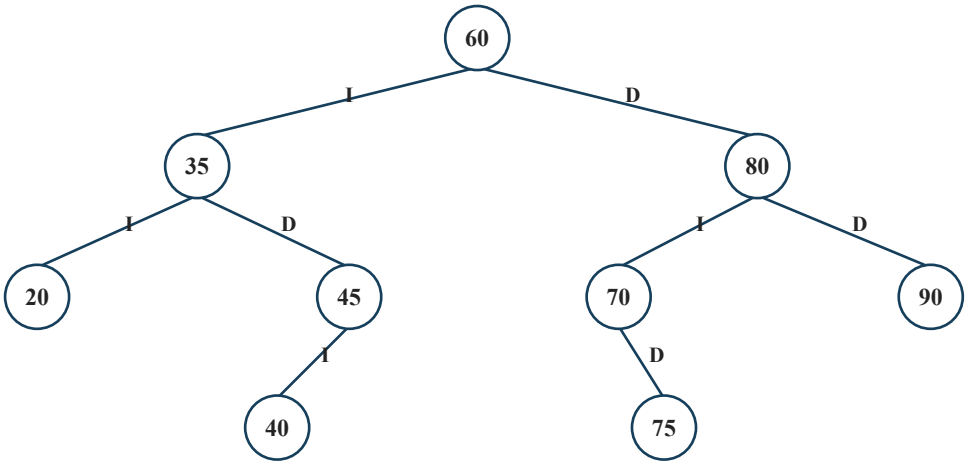
El código de un libro debe ser positivo y no se puede cambiar después de creado. El título y el autor no pueden estar vacíos, y el año debe estar entre 1900 y 2026. Los códigos de las unidades se guardan en mayúsculas, así que una búsqueda como “sw” encuentra la unidad “SW”.

3. La biblioteca de ejemplo

```
BIB Biblioteca FISEI
  SW Software
    SW-E1 Estante 1 de Software: 60, 35, 80, 20, 45, 70, 90, 40, 75
    SW-E2 Estante 2 de Software: 15, 10, 25
  TI Tecnologías de la Información
    TI-E1 Estante 1 de TI: 50
  TEL Telecomunicaciones
```

La estructura tiene siete unidades, cuatro hojas y una altura de dos aristas. Entre los tres estantes hay trece libros. En el ABB de SW-E1, por ejemplo, 35 queda a la izquierda de 60 porque su código es menor; 80 queda a la derecha porque es mayor.

Árbol inicial del estante SW-E1 (I: hijo izquierdo, D: hijo derecho)


60
I
D
35
80
I
D
I
D
20
45
70
90
I
D
40
75

4. Recorridos y traza del ABB

El catálogo de SW-E1 contiene los códigos insertados en el orden indicado por la consigna. Los tres recorridos visitan los mismos nodos, pero cambian el momento en que procesan cada nodo.

```
Preorden: 60 35 20 45 40 80 70 75 90
Inorden:  20 35 40 45 60 70 75 80 90
Posorden: 20 40 45 35 75 70 90 80 60
```

Traza del recorrido preorden

```
entra 60
VISITA 60
  entra 35
  VISITA 35
    entra 20
    VISITA 20
      null -> regresa
      null -> regresa
    entra 45
    VISITA 45
      entra 40
      VISITA 40
        null -> regresa
        null -> regresa
      null -> regresa
  entra 80
  VISITA 80
    entra 70
    VISITA 70
      null -> regresa
      entra 75
      VISITA 75
        null -> regresa
        null -> regresa
    entra 90
    VISITA 90
      null -> regresa
      null -> regresa
Secuencia PREORDEN: 60 35 20 45 40 80 70 75 90
```

5. Inserción y eliminación

La inserción de un código repetido se rechaza. Para comprobar las tres situaciones de eliminación, se inserta 42 y se eliminan 20 (hoja), 70 (un hijo) y 60 (dos hijos), en ese orden. Al retirar un nodo con dos hijos, se utiliza el sucesor inorden, el menor código del subárbol derecho, para mantener el orden del ABB. El preorden final es 75 35 45 40 42 80 90.

6. Pruebas y reflexión

La compilación local con JDK 17 finalizó correctamente. La ejecución actual de PruebasBiblioteca terminó con 13/13 pruebas OK; los nombres y el resultado exacto se conservan en docs/resultado-pruebas.txt.

```
OK  biblioteca con solo la raíz
OK  estructura inicial
OK  niveles y búsqueda sin distinguir mayúsculas
OK  reglas de inserción y códigos únicos
OK  recorridos del estante SW-E1
OK  duplicado, camino y rango
OK  eliminación de hoja, un hijo y dos hijos
OK  modificación conserva código y valida cambios
OK  operaciones sobre libro inexistente
OK  validaciones de datos de libro
OK  ABB inválido por nieto mal ubicado
OK  reconstrucción directa desde preorden
OK  inserción de tres códigos demostrativos
13/13 pruebas OK
```

Reflexión

¿Por qué buscar en el ABB es O(h) y en el árbol general es O(N)? ¿Qué pasa si se insertan códigos ordenados?

En el ABB, cada comparación permite continuar por un solo hijo, por lo que se recorren como máximo h niveles. En el árbol general no hay una regla de orden que descarte ramas y, en el peor caso, se revisan N unidades. Si se insertan códigos ya ordenados en un ABB sin balanceo, el árbol puede quedar como una cadena: h = N - 1 y la búsqueda pasa a O(N).

¿Por qué se usa el sucesor inorden al eliminar un nodo con dos hijos?

El sucesor es el menor valor del subárbol derecho. Es mayor que todos los valores del subárbol izquierdo y no supera a los demás del derecho; por eso puede ocupar el lugar eliminado sin romper la propiedad del ABB. Un nodo arbitrario no garantiza ese orden.

¿Cómo se aplicarían estos árboles en SmartETL?

Un árbol general puede representar proyectos, fuentes, procesos y destinos con relaciones jerárquicas. Un ABB puede mantener registros identificados por un código y facilitar búsquedas y recorridos ordenados. En un sistema con datos reales también habría que comparar esta solución con índices de una base de datos y considerar el balanceo y la concurrencia.

7. Repositorio, conclusiones y entrega

El código está publicado en el repositorio público Deber-Arboles-VillacresFernando. El pull request #1, de desarrollo a main, fue integrado el 22 de octubre de 2026. La rama main contiene la implementación y el workflow de GitHub Actions terminó correctamente.

Conclusiones

La práctica permitió implementar dos estructuras con responsabilidades distintas: el árbol general organiza las áreas y estantes, y cada ABB ordena los libros por código. Las pruebas confirmaron las reglas de inserción, búsqueda, recorridos, modificación y eliminación, incluida la sustitución por sucesor inorden.

También se verificó que la eficiencia del ABB depende de su altura. La recursividad hizo explícito el recorrido de cada estructura y las pruebas ayudaron a comprobar casos límite, como los datos inválidos y un valor mal ubicado en un nivel profundo.

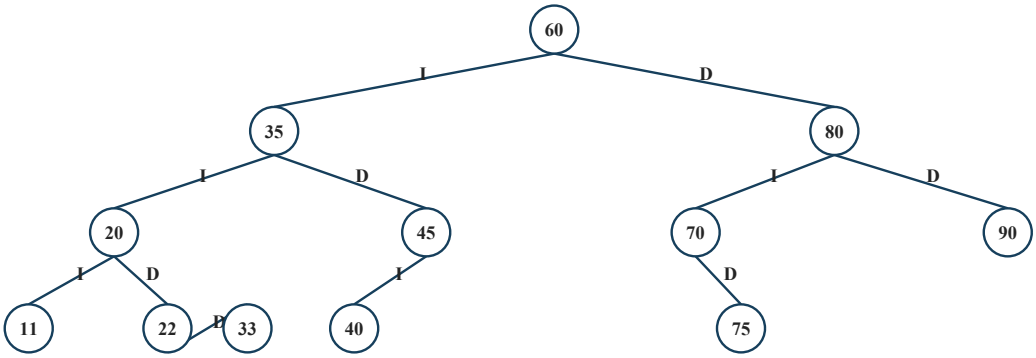
Antes de subir el PDF a Moodle

Las trazas del anexo usan 11, 22 y 33 únicamente como ejemplo; no son códigos personales calculados desde una cédula. Deben sustituirse por los tres valores correctos. La consigna también pide capturas auténticas de la ejecución, del historial de commits y del pull request. Este informe no las inventa ni las presenta como adjuntas; agrégalas antes de entregar.

Actualización de la implementación y las pruebas

Esta página complementa el informe con una prueba adicional del ABB que reproduce las inserciones de tres códigos de ejemplo. Los valores 11, 22 y 33 son demostrativos; no se derivaron de datos personales. Sustituirlos por los códigos calculados por el estudiante antes de presentar las trazas como personales.

Árbol demostrativo de SW-E1



I: hijo izquierdo; D: hijo derecho.

Recorridos con 11, 22 y 33 agregados al final

```
Preorden: 60 35 20 11 22 33 45 40 80 70 75 90
Inorden:  11 20 22 33 35 40 45 60 70 75 80 90
Posorden: 11 33 22 20 40 45 35 75 70 90 80 60
```

Verificación actual

Se amplió PruebasBiblioteca para crear este árbol mediante DatosPrueba.crearCatalogoSW_E1ConCodigos(11, 22, 33). La nueva suite incluye trece casos y comprueba el tamaño, el preorden y la validez ABB del árbol demostrativo.

```
OK  biblioteca con solo la raíz
OK  estructura inicial
OK  niveles y búsqueda sin distinguir mayúsculas
OK  reglas de inserción y códigos únicos
OK  recorridos del estante SW-E1
OK  duplicado, camino y rango
OK  eliminación de hoja, un hijo y dos hijos
OK  modificación conserva código y valida cambios
OK  operaciones sobre libro inexistente
OK  validaciones de datos de libro
OK  ABB inválido por nieto mal ubicado
OK  reconstrucción directa desde preorden
OK  inserción de tres códigos demostrativos
13/13 pruebas OK
```

La salida exacta también está guardada en docs/resultado-pruebas.txt. Los códigos personales y las capturas de pantalla auténticas deben completarse por el estudiante; esta actualización no los sustituye.